

TEAM 404 · ROUND 2

AutoFleet AI

Autonomous last-mile disruption resolution

Team Plan — the idea, the tech, and what to build

4 people · ~10 days

github.com/Himanshi314/AUTOFLEET

Read this whole document before writing any code. Part 1 is the idea, Part 2 is how the tech actually works, Part 3 is your specific tasks with a definition of "done".

PART 1 — THE IDEA

One sentence

When a delivery goes wrong, a human has to fix it, and while they're fixing it the delivery fails. AutoFleet AI fixes it immediately instead.

Everything else is elaboration on that. If any part of the pitch can't get back to that sentence, we've drifted.

The story — all four of us must be able to tell this without notes

It's 7pm. Arjun is 3 km from Rohan's flat with a parcel. His bike dies.

Today: Arjun calls the hub. The coordinator is on another call. Eight minutes pass. They start working down a list of riders — there's one 600 m away, but they don't know that. They call a rider who's mid-delivery. They call another. Rohan has been told nothing.

At 7:40 the window closes. The parcel goes back to the warehouse and comes out again tomorrow — a second trip, a second attempt, an annoyed customer, and Arjun loses the earnings for a delivery he didn't complete, through no fault of his own.

Time to fix: ~20 minutes. Time available: ~8. That gap is the product.

What we are NOT solving

Boundaries are a strength. Do not let scope drift.

- ❌ Not route planning — Google Maps and every TMS already do that
- ❌ Not initial dispatch — dispatch systems do that
- ❌ Not demand forecasting or fleet optimisation
- ✅ **Only** the 10–20 minutes *after* the plan has already broken

THE NEW FRAMING — severity decides which agents run

This is the change from what we pitched in round 1, and it's the most important idea in the project now.

Our deck (slide 3) says "**relevant AI agents activated.**" Not *all* agents. The code now does exactly that — `autofleet/routing.py` decides per incident.

The fix, and the insight behind it:

Assessing how serious a disruption is only matters if it changes which agents run. Otherwise severity is just a sentence in a card — theatre.

So severity becomes a **router**, not a label. And critically, the routing decision costs nothing: everything needed to make it is known *before* any AI call.

Situation	Agents that run	AI calls
Internal-only fix, ETA barely moves (a scheduling clash)	none — the models resolve it outright	0
Recipient's expectations change, courier keeps the job	Risk → Customer → Communication → Coordinator	4
Courier disabled, or a better driver now outranks them	all six	6
Payload not deliverable — replacement from depot	all six	6
No eligible driver at all	none — escalate to a human	0

An ETA shift counts as significant if it exceeds **20% of the current ETA, floor 5 minutes** — six minutes on a 70-minute run is noise; six on a 12-minute run is most of the journey.

Three things this buys us that we currently cannot claim:

1. **Slide 3 becomes literally true.**
2. **Cost story on screen.** Untiered, 20,000 exceptions/day ≈ \$1,360/day. Routed, ≈ \$180/day. A tile reading `LLM calls saved: 47` makes that visible.
3. **A better definition of autonomy** — a system that decides *how much intelligence a problem deserves* is more autonomous than one that fires everything at everything.

Measured across all 28 disruption × delivery combinations: 29% full chain, 61% partial, 11% deterministic.

The corrected agent lineup

Our round-1 deck named five agents. The code used different names. **The code now matches the deck**, because judges who saw round 1 may check.

New chain (this matches deck slide 7, with Risk moved to the front):

```
[ router picks which of these actually run ]
|
1. 🕒 Risk Agent           how severe is this, and what kind of problem
2. 👤 Customer Agent      what we ask of the recipient
3. 🗣️ Communication Agent what message actually goes out
4. 📦 Resource Agent       which driver takes it (also sees route alternates)
5. 🚚 Delivery Agent       original courier's status, support, earnings
6. 🧠 Coordinator Agent    the final authoritative resolution
```

Mapping from the old code, so nobody gets lost:

Deck name	Old code name	Action
Customer Agent	Customer Agent	unchanged
Delivery Agent	Driver Agent	rename
Resource Agent	Reallocation Agent	rename
Risk Agent	<i>(didn't exist as an agent)</i>	add , goes first
Communication Agent	<i>(folded into Customer)</i>	add , split out

Deck name	Old code name	Action
Coordinator Decision	Coordinator Agent	unchanged
—	Route Agent	remove — its route data becomes Resource Agent's input

Why Risk goes first

You can't decide what to tell the customer until you know whether this is a delay or a hard failure. Severity has to come before every decision that depends on it.

The Risk Agent, precisely

Its name covers two different jobs, and we split them:

- **Detect** — "*is something going wrong?*" Pure arithmetic: 7 features × 7 weights. Runs every 2.2s on every delivery. **No AI.** This is already built and it's our strongest technical feature — it's what lets the system fire before a human notices.
- **Assess** — "*how bad is it, what kind of problem is this?*" Reads those numbers and forms a judgement that constrains everyone downstream. **This is the Risk Agent, now built.**

What it outputs — this is real output from a run:

"Failure risk on D-102 is 55% (elevated), driven mainly by corridor congestion compounding with schedule pressure. This is a hard stop rather than a delay — the courier cannot continue, so reassignment is mandatory and no same-courier retry should be offered."

Note the last clause is an **instruction to the agents after it**. That's the job.

Do not turn detection into an LLM agent. Keep the model. Say this out loud in the pitch: "*the Risk Agent is backed by a scoring model rather than a prompt — that's why it can fire before a human notices.*" That's an upgrade over the deck, not a gap.

PART 2 — HOW THE TECH ACTUALLY WORKS

An "agent" is not a program. It's a text message.

There is no agent object running anywhere. An agent is:

a block of instructions + a block of data, sent to Claude over the internet, and some text comes back.

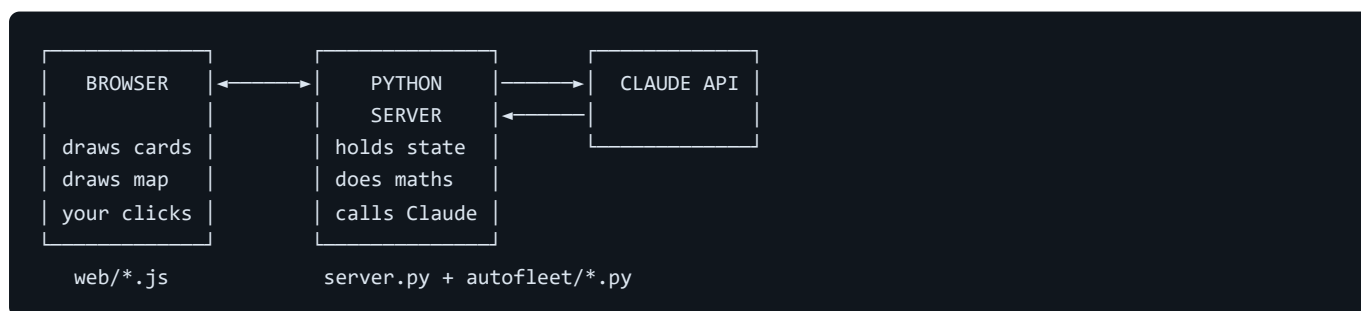
That's the whole thing. Run this to see a real one with your own eyes:

```
python -X utf8 -c "  
from autofleet import llm as L  
from autofleet.world import World  
from autofleet.agents import run_chain  
grabbed=[]  
o=L.LLM.stream  
def spy(self,*,system,user,fallback,force_fallback=False):  
    grabbed.append((system,user)); return o(self,system=system,user=user,fallback=fallback)  
L.LLM.stream=spy  
run_chain(World(), L.LLM(), delivery_id='D-102', disruption_key='bike_breakdown', emit=lambda e:None)  
s,u=grabbed[0]; print('--- INSTRUCTIONS ---'); print(s); print('--- DATA ---'); print(u)"
```

"Five agents" = sending that five times with five different instruction paragraphs. Same code, same model, same API call. Different paragraph. The word "agent" makes it sound like a creature; it's a job description stapled to a text message.

They share **no memory**. Every call sends `messages=[{one user turn}]` — no conversation history. What carries forward is a Python list we paste into the next prompt as text. So it's not one agent wearing five hats (a hat-wearer would remember). It's **five briefings to five specialists who never meet**, each handed the previous ones' conclusions and nothing else.

The three boxes



- **Browser** is dumb — draws what it's told, reports clicks.
- **Python server** is the real brain — holds deliveries/drivers in dictionaries, does every calculation, decides when to call Claude.
- **Claude API** is a website you send text to. Stateless. Remembers nothing.

The surprise: most of this is NOT AI

Understand this table or you cannot defend the project.

File	What it does	AI?
<code>geo.py</code>	Distance between two GPS points (haversine — school trig)	✗
<code>scoring.py</code>	Multiply features by weights, add them up	✗
<code>impact.py</code>	Multiply km × emission factor	✗
<code>world.py</code>	Dictionaries holding drivers and deliveries	✗
<code>server.py</code>	A web server	✗
<code>agents.py</code>	Builds text, sends it, reads the reply	✓ only here

What Claude actually does in the whole system: writes ~6 sentences, and picks one driver ID from a list that was already ranked for it. That's it.

That's not a weakness — it's the correct design, and it's why the system still works when the AI fails. **The intelligence is in the decomposition, not the model.**

The two models (this is our ML story)

Both in `autofleet/scoring.py`. Both linear and interpretable **on purpose** — every score breaks down into per-feature contributions that the UI renders. That breakdown is what earns operator trust; don't trade it for a fancier model.

`disruption-risk-v1` — logistic, 7 features, `BIAS = -4.60`. Predicts probability a delivery fails, so we can act before it does.

Feature	Weight
recipient_absence_rate	2.60
address_uncertainty	2.35
vehicle_health_risk	2.10
traffic_index	1.95
schedule_pressure	1.70
driver_fatigue	1.45
weather_risk	1.15

Bands: `critical ≥ 0.68` (same line the watchdog acts on), `elevated ≥ 0.42`, `watch ≥ 0.15`.

`reassignment-suitability-v1` — weighted linear, weights sum to 1.0. **Hard constraints run first and are absolute** (cold-chain capability, shift time, capacity, payload size, cold-chain deadline). Only survivors get scored.

Feature	Weight
proximity	0.34
eta_fit	0.20

Feature	Weight
reliability	0.15
load_headroom	0.11
shift_headroom	0.09
capability_margin	0.07
zone_familiarity	0.04

Neither is trained. Both weight sets are hand-set from domain reasoning. The dashboard says so. **Never claim we trained them.** If asked what *would* be trained: the risk model is supervised with free labels (did the delivery fail on first attempt?); the ranker is learning-to-rank with a selection-bias problem (you only see outcomes for drivers actually chosen).

What happens when you click the button — full trace

1. You click `Vehicle Breakdown` on D-102.
2. Browser sends `POST /api/disrupt {delivery_id, disruption}` — a normal web request.
3. `server.py` receives it, puts it on a queue.
4. A background thread picks it up, calls `run_chain()`.
5. **Plain Python maths runs first** — risk score, incident details gathered into JSON.
6. **The router decides which agents to run** (new work — see Track A/B tasks).
7. **Agent fires** — server builds the text block, sends it to Claude, waits.
8. **Text returns a few letters at a time** (streaming — that's why it types on screen).
9. Each fragment is pushed to the browser down an open connection; `app.js` appends it.
10. **More maths** — `route_alternates()`, `RANKER.rank()`. No AI.
11. **★ The action happens.** `apply_resolution()` changes the dictionaries: driver swapped, original marked unavailable, ETA changed, impact recorded. **This is the most important line in the project** — something in the world changed, not just words on screen.
12. Coordinator describes what happened.
13. Browser draws the green banner.

Architecture properties worth knowing

- **One SSE channel** carries everything. This is why the watchdog can push a chain the browser never asked for — that's what makes the autonomy real rather than a browser timer.
- **Incidents are queued and serialised** on one worker thread, so rapid clicks don't interleave. Demo choice, not a limit.
- **Chains carry a world generation.** Reset or scenario-switch mid-incident aborts the chain cleanly with no partial decision applied.
- **The chain always terminates.** Three layers: 25s per-call timeout, 45s chain budget (past it, agents use deterministic fallbacks), and a worker `finally` that frees any delivery left in `Resolving`. Worst case ~50s, bounded.

- **The dependencies are a DAG, not a cycle.** Risk → Customer → Communication → Resource → Delivery → Coordinator is a genuine order. **There is nothing to negotiate**, which is why a pipeline is correct and agent-to-agent negotiation would be a downgrade.

Live vs simulated

Mode	How	Behaviour
Live	<code>ANTHROPIC_API_KEY</code> in <code>.env</code> + <code>pip install anthropic</code>	Agents stream from Claude Opus 5. Header: <code>LIVE</code> · <code>claude-opus-5</code>
Simulated	nothing needed	Same chain, same models, deterministic text. Every card labelled <code>simulated</code>

Simulated mode is the demo safety net — **not** a way to pass canned text off as model output. **Never remove the labelling.**

⚠️ **The live path is written but has never run with a real key.** Track A's first job.

PART 3 — WHAT TO BE DONE

Ground rules

- 1. **You must be able to explain every line in your track.** A judge asking "why 0.68?" and getting silence ends the demo. Ownership = comprehension.
- 2. **Nobody edits another track's files.** Ask the owner. Tracks are deliberately file-disjoint so we don't fight over merges.
- 3. **One branch per track.** Team lead merges to `main` at Day 6 and Day 9 only.
- 4. **Commit small, commit daily.**
- 5. **15-minute standup, same time, every day.** What I did / what's next / what's blocking me.
- 6. **By Day 8 every person can run the whole demo alone.** If one of us is ill on presentation day, we're still fine.

File ownership

Track	Owner	Files — yours alone
A · Agents & Pitch	team lead	autofleet/agents.py , autofleet/llm.py
B · Models & Routing		autofleet/scoring.py , autofleet/impact.py , autofleet/geo.py , new: autofleet/routing.py
C · Frontend & Demo		web/index.html , web/styles.css , web/app.js
D · Backend & Reliability		server.py , autofleet/world.py

TRACK A — Agents & Pitch

Your judge question: "How is this not just a chatbot wrapper?"

#	Task	Done when
A1	Get live agents working. Key in <code>.env</code> , <code>pip install anthropic</code> , run a chain.	Header shows <code>LIVE · claude-opus-5</code> and cards say <code>live</code> , not <code>simulated</code> . Do this on Day 1 — if it's blocked we must know immediately.
A2 ✓	DONE — Rename agents to match the deck. <code>Driver Agent</code> → <code>Delivery Agent</code> , <code>Reallocation Agent</code> → <code>Resource Agent</code> . In <code>AGENT_SPECS</code> and <code>_ROLE_PROMPTS</code> .	Dashboard shows the deck's names.
A3 ✓	DONE — Remove the Route Agent; pass its <code>route.alternates</code> output into the <code>Resource Agent</code> 's input instead.	Chain has no <code>Route</code> card; <code>Resource Agent</code> 's prompt contains <code>route alternates</code> .

#	Task	Done when
A4 ✓	DONE — Add the Risk Agent , first in the chain. Prompt: read the risk score + contributions, state severity, and state what it means for the agents after it.	A Risk card appears first and its text constrains the later agents.
A5 ✓	DONE — Add the Communication Agent after Customer. Customer decides <i>what we ask of the recipient</i> ; Communication decides <i>what message goes out</i> .	Six agent cards; the two decisions are visibly different, not duplicates.
A6 ✓	DONE — Wire in the router — call <code>routing.plan_chain()</code> (Track B builds it) and only run the agents it returns.	A <code>customer_not_home</code> on a healthy delivery runs 0–4 agents, not 6.
A7	Real tool use for the Resource Agent's pick. Replace the <code>PICK:</code> regex with a schema-validated tool call.	No regex parsing; the model returns structured JSON. Kills our weakest architectural point.
A8	Coordinator fact-check. Assert every number in the Coordinator's output appears in its input; fail loudly if not.	A "facts verified ✓" signal per incident.
A9	Write the pitch + Q&A prep doc. 10 likely questions with answers. Must include: "where's the ML?", "so it's just prompts?", "what if it fails?", "what's your weakest part?"	A doc all four of us have rehearsed.

TRACK B — Models & Routing

Your judge question: "Where's the actual ML, and where do these numbers come from?"

#	Task	Done when
B1 ✓	DONE (needs your tests) — <code>autofleet/routing.py</code> — the severity router. One pure function: <code>plan_chain(disruption, risk, ranking) -> list[agent_id]</code> . Uses only facts known before any AI call: <code>disables_driver</code> , <code>severity</code> , risk band, <code>margin_over_next</code> , <code>needs_replacement_stock</code> . This is the most important task on the team — it makes slide 3 true.	Returns the right agent list for each row of the routing table in Part 1. Unit-test each row.
B2 ✓	DONE — <code>llm_calls_saved</code> — how many agent calls the router skipped vs. running all six. Track it cumulatively for the impact tile.	The number is available for Track C to render.
B3	Verify every emission factor against its real source. Look up the actual DEFRA/BEIS figure for a petrol two-wheeler and the CEA grid factor for India. Fix what's wrong. Cite properly in <code>impact.py</code> .	Every factor in the Assumptions drawer has a source you personally checked. You are the person who can say "I verified these."
B4	Write the eval scaffold — a small script scoring the Resource Agent's driver pick against a known-good answer. Doesn't need real data; needs to exist and run.	<code>python eval.py</code> prints a score.
B5	Own the model cards. Be able to explain every weight, why the model is linear, why nothing is trained, and what <i>would</i> be trained.	You can answer the ML question cold, with no notes.

TRACK C — Frontend & Demo

Your job: the first 10 seconds, and making sure the demo cannot die.

#	Task	Done when
C1	Actually look at the UI. It was built and verified structurally but nobody has seen it with human eyes . Open it, check every panel.	You've listed every visual defect you found.
C2	Check it on the real presentation screen/projector. Dark themes wash out badly on cheap projectors.	Contrast confirmed readable on the actual hardware. If not, lighten the palette.
C3 ✓	DONE — agent names in the UI to match Track A's renames, and make room for six cards instead of five.	Six cards render cleanly without scrolling problems.
C4 ✓	DONE — the AI calls avoided tile using Track B's number.	Tile ticks up as incidents are routed.
C5 ✓	DONE — Show skipped agents. When the router skips an agent, show it greyed out with the reason — <i>"skipped: no reassignment needed"</i> . This is how the judge SEES the routing decision. Without it, the smartest part of the system is invisible.	Skipped agents visibly appear as skipped, not absent.
C6	Write the demo choreography — exactly what's on screen at each beat, and what the presenter says.	A written script, timed, under 4 minutes.
C7	Record a backup video of a perfect run.	An MP4 on the presenting laptop. If the network dies on stage, we play it and keep talking.

TRACK D — Backend, Scenarios & Reliability

Your judge question: *"What happens when it breaks?"*

#	Task	Done when
D1 ✓	DONE — the 2 missing disruptions from deck slide 2: <code>conflicting_assignment</code> and <code>priority_override</code> . Add to <code>DISRUPTIONS</code> with <code>disables_driver</code> , <code>severity</code> , <code>detected_as</code> , etc.	Both appear as buttons and both resolve end to end.
D2	Make the demo state resettable and repeatable. Reset must always return to a clean, identical starting state.	Press Reset 10 times, get the same board every time.
D3	Break it on purpose. Kill the network mid-chain. Use a bad API key. Trigger 6 incidents at once. Switch scenario mid-chain.	You've written down what happens in each case and nothing leaves a delivery stuck.
D4	Decide and set up hosting. Localhost on the presenting laptop is a legitimate and often safer choice.	Decision made and tested on the actual machine.
D5	Own the simulated-mode safety net. It must be flawless and indistinguishable in polish, because it's what runs if the API is down.	Full demo runs perfectly with no API key.

Everyone, in the first two days

#	Task
E1	Clone, run <code>python server.py</code> , see it working.
E2	Read this whole file, plus <code>CLAUDE.md</code> .
E3	Run the "see a real agent prompt" command in Part 2. Understand that an agent is a text message.
E4	Answer these 8 questions independently, in writing, in your own words. Then compare all four sets. Wherever we disagree is where the idea is still fuzzy — that's our real to-do list.
E5	Explain your track out loud to the group for 5 minutes, no notes. This is the Day-2 gate.

The 8 questions: 1. What goes wrong, in one sentence? 2. Who fixes it today, and how long do they take? 3. What does it cost when nobody fixes it in time? 4. What does our system do instead? 5. Why can't you just write rules for this? 6. Who would pay for it, and what line on their budget does it reduce? 7. What does our system refuse to do on its own? 8. What's the weakest part of our idea?

Answers are in `CLAUDE.md` section 2 — **but write yours first, then compare.** Reading them isn't the same as owning them.

E6 — the highest-value task on this entire list

One of us talks to a real delivery rider. Ten minutes. A Delhivery, Ecom, Zepto or Porter rider — they're everywhere. Three questions:

1. What happens when you can't deliver something?
2. Who do you call, and how long do you wait?
3. What happens to your money when a delivery fails?

Costs nothing. It converts every answer above from something we read into something we know — and that difference is audible to a judge. **We currently have never observed the problem we're solving.** That's weakness #3 on our own list.

The 10 days

Days	Phase	Gate
1–2	Own it. Everyone runs it, reads their files, makes one real change. A1 (live API) done.	Each person explains their track for 5 min, no notes.
3–5	Build round 1. Deck alignment (A2–A6, B1, D1) + C1/C2.	All tracks' work runs on <code>main</code> .
6	Integrate + dry run #1. Full demo, timed, someone playing hostile judge.	We find what's broken with 4 days left.
7–8	Build round 2. Only what the dry run exposed. No new ideas.	Dry run #2 clean. Backup video recorded.
9	FREEZE. No code. Rehearse ×3.	Everyone can demo solo.
10	Buffer. Something will go wrong; this day is for that.	—

Integrating on Day 6 rather than Day 9 is the highest-value decision in this plan. Most teams integrate the night before and discover their pieces don't fit.

The Day 9 freeze is not negotiable. More demos are lost to a last-minute "small improvement" than to missing features.

Say no to

- ❌ A third or fourth scenario
- ❌ Real map tiles (adds a network dependency on stage; breaks self-containment)
- ❌ Login, accounts, a database
- ❌ Agent-to-agent negotiation (see: the DAG argument in Part 2)
- ❌ Training a model — we have no labelled data and 10 days

Add depth, not surface. One well-defended model beats three shallow features.

Our weakest points — know these, they're an asset

Being able to name your own weaknesses is the clearest sign you understand a project. Most teams can't do it. Rehearse these:

1. **Rules could handle most of it.** Our own measurement: 66% of exceptions have one obviously best driver. The AI layer only pays for itself on the ambiguous tail, and we haven't proven that tail is big enough with real data.
2. **Nothing is trained.** Both models are hand-calibrated. The dashboard admits it.
3. **We've never seen the real problem.** The 10–20 minute figure comes from research, not observation. (*Fix with E6.*)
4. **Trust is the barrier, not the tech.** Real adoption needs shadow mode → assisted → guarded → autonomous. We've built only the last stage.
5. **The agent chain isn't defensible.** A weekend's work for an incumbent. What would compound is the policy library, the integrations, and a shadow-mode record proving the decisions hold up. We have none of those.

The strongest version of our pitch is the honest one:

Rules handle the head of the distribution. Nobody can write rules for the tail — the case where the nearest driver is 0.4 km away but their shift ends in 20 minutes and the payload needs a cold box and the customer already rescheduled twice. That tail is where coordinators spend their day. The models resolve the routine two-thirds; the agents cover the tail without anyone writing 400 rules.